
Hito 3

Integracion Final, Escalabilidad y Validacion Integral

CleanPool App

Equipo: Cleanpool

Seccion: 300

Entrega: 20 de mayo de 2026

Integrantes: Vicente Araya (PM) | Diego Carmona (SM) |
Daniel Cortez | Constanza Malebran

Índice

1. Estado General del Proyecto	3
1.1. Resumen Ejecutivo	3
1.2. Objetivos Alcanzados	3
1.3. Estado de Madurez del Sistema	3
2. Arquitectura Final del Sistema	4
2.1. Arquitectura Integrada	4
2.2. Integración de Componentes	4
2.3. Escalabilidad y Rendimiento	4
3. Validación Funcional y Técnica	5
3.1. Funcionalidades Finales Implementadas	5
3.2. Casos de Uso Validados	5
3.3. Pruebas Realizadas	6
3.4. Cobertura y Calidad	6
4. Despliegue y Operación	6
4.1. Infraestructura de Producción	6
4.2. Proceso de Despliegue	7
4.3. Monitoreo y Logs	7
5. Gestión Scrum y Trabajo Colaborativo	7
5.1. Estado Final del Sprint	7
5.2. Commits y Participación	7
5.3. Problemas y Resoluciones	8
6. Demostración de Valor	8
6.1. Valor Entregado al Usuario	8
6.2. Evidencia Funcional	8
6.3. Retroalimentación Obtenida	8
7. Conclusiones y Próximos Pasos	9
7.1. Conclusiones Técnicas	9
7.2. Aprendizajes del Equipo	9
7.3. Trabajo Futuro	9
A. Capturas del Sistema	10
B. Repositorio y Evidencias	13
C. Commits Relevantes	13

1. Estado General del Proyecto

1.1. Resumen Ejecutivo

El sistema **CleanPool App** integra monitoreo IoT para piscinas residenciales mediante sensores que publican lecturas (ej. temperatura y ORP) hacia un broker MQTT. El **frontend** (Flutter) consulta y visualiza el estado de las piscinas, mientras que el **backend** (FastAPI) expone endpoints protegidos (JWT) para la gestión de usuarios y piscinas.

En el contexto de Hito 3, el foco de integración final se centra en: (i) consolidar el flujo completo desde la ingestión de datos IoT hasta la evaluación del estado (priorizando sensor sobre mantenimiento manual) y (ii) asegurar que el despliegue sea repetible mediante contenedores Docker y automatización con GitHub Actions / Dokploy.

1.2. Objetivos Alcanzados

Objetivo funcional: Se implementan y validan flujos de autenticación JWT, CRUD de piscinas, cálculo y persistencia de tratamientos manuales, registro histórico de mantenimientos y evaluación de estado de piscina usando reglas que priorizan lecturas de sensores frescas sobre datos manuales.

Objetivo técnico: Se consolida una arquitectura de despliegue con **Docker Compose** (MongoDB, Mosquitto MQTT, backend FastAPI y frontend), y se integra un worker MQTT en segundo plano que suscribe y persiste lecturas en MongoDB. Adicionalmente, se incorporan **logs estructurados** persistidos en MongoDB para trazabilidad de requests.

Objetivo de negocio: Se entrega una solución operativa que reduce la dependencia de verificación manual esporádica, permitiendo al usuario visualizar el estado de aptitud y mantener trazabilidad de mantenimientos.

1.3. Estado de Madurez del Sistema

Componente	Estado actual
Frontend	Operativo: app Flutter con consumo REST (login, CRUD de piscinas, estado y mantenimientos). Evidencia visual incluida en el Anexo A.
Backend	Operativo: FastAPI con endpoints para autenticación, gestión de piscinas, lectura del estado (sensor >manual), mantenimientos, tratamiento manual y ingestión IoT.
Base de datos	Operativo: MongoDB en contenedor (persistencia de colecciones como <code>pools</code> , <code>mantenciones</code> , <code>lecturas</code> y <code>device_bindings</code>).
Integraciones	Operativo: Mosquitto MQTT (topics de sensores) con ingestión hacia MongoDB, utilizando API key para proteger la ingestión HTTP legacy.
Infraestructura	Operativo: stack reproducible con Docker Compose y despliegue automático (Dokploy y/o workflow de CI/SSH).

Testing	Operativo: se ejecutan pruebas backend con <code>pytest</code> (y pruebas frontend con <code>flutter test</code> en CI).
---------	--

2. Arquitectura Final del Sistema

2.1. Arquitectura Integrada

Frontend (Flutter)	Backend (FastAPI)
Consume endpoints REST: <code>/auth/...</code> , <code>/piscinas/...</code> , <code>/mantenciones/...</code> , <code>/piscinas/{id}/status</code> .	Exposicion y validacion de endpo
MongoDB (Atlas / contenedor)	Mosquitto MQTT
Persistencia: piscinas, mantenciones, lecturas y historial/logs.	Broker para lecturas IoT (topics

Flujo principal: Sensor IoT publica hacia Mosquitto → worker MQTT persiste lecturas en MongoDB → backend evalua estado (sensor frescos >manual) → frontend muestra estado y recomendaciones.

2.2. Integracion de Componentes

Componente	Integracion realizada	Resultado
Frontend (Flutter)	Consumo de APIs REST, gestion de sesion JWT y visualizacion del estado de piscina.	Operativo (ver Anexo A).
Backend (FastAPI)	Endpoints para auth, CRUD de piscinas, status unificado (sensor >manual) y mantenciones; worker MQTT en segundo plano.	Operativo.
Base de datos (MongoDB)	Persistencia de colecciones operativas (<code>pools</code> , <code>mantenciones</code> , <code>lecturas</code> , <code>device_bindings</code>) y logs estructurados.	Operativo.
APIs / ingestion IoT	Ingestion MQTT (worker) y endpoints legacy protegidos (API key).	Parcial/operativo segun via (MQTT y REST).

2.3. Escalabilidad y Rendimiento

Aspecto	Implementacion	Impacto esperado
Contenedores	Docker Compose separa servicios (MongoDB, Mosquitto, backend, frontend).	Facil despliegue y escalamiento horizontal por servicio.
Ingestion IoT	Worker MQTT ejecuta en hilo en background y persiste lecturas en MongoDB.	Reduccion de latencia percibida en endpoints REST.
Evaluacion sensor vs manual	Regla <code>is_reading_fresh</code> utiliza ventana de frescura (2 minutos) para decidir fuente confiable.	Evita resultados obsoletos y mejora estabilidad del estado.
Logs y trazabilidad	Middleware mide latencia request y persiste logs estructurados en MongoDB.	Mejora monitoreo y debugging para mantener calidad operativa.

3. Validacion Funcional y Tecnica

3.1. Funcionalidades Finales Implementadas

ID	Historia de usuario	Descripcion	Estado
HU-01	Auth	Registro/login y acceso autenticado (JWT) y datos del usuario via <code>/auth/me</code> .	Completa
HU-02	Gestion de piscinas	CRUD de piscinas y consulta de estado de piscina (<code>/piscinas/{id}/status</code>), aplicando priorizacion sensor <code>>manual</code> .	Completa
HU-03	Ingestion IoT y mantenimiento	Ingestion de lecturas IoT (MQTT) y consolidacion de mantenimientos/tratamientos con historial.	Completa
HU-04	Mantenciones	Registro y consulta del historial de mantenimientos (<code>/mantenciones/</code>) y persistencia.	Completa

3.2. Casos de Uso Validados

Se validaron los siguientes casos de uso en el flujo principal del sistema:

- El usuario autentica via JWT, obtiene su perfil (`/auth/me`) y gestiona sus piscinas (`/piscinas`).
- El usuario consulta el estado de una piscina con `/piscinas/{id}/status`. Cuando existen lecturas de sensor dentro de la ventana de frescura, el estado utiliza esos valores; si no, se apoya en mantencion manual mas reciente.
- Se registra mantencion manual (`/piscinas/{id}/tratamiento`) y se revisa historial

(/mantenciones/).

- Se valida la ingestión IoT mediante MQTT: el backend suscribe los topics configurados y persiste lecturas en MongoDB, afectando el estado posterior.

3.3. Pruebas Realizadas

ID	Tipo	Descripción	Resultado
TP-01	Unitaria	Validación de servicios (ej. calculadora de aptitud, par-ser/validaciones) mediante <code>pytest</code> .	Pasa
TP-02	Integración	Flujo end-to-end de integración backend (auth, CRUD piscinas, status, mantenciones) con mocks y/o DB simulada.	Pasa
TP-03	E2E	Ejecución manual guiada por swagger/curl según <code>MANUAL_DE_PRUEBAS.md</code> para verificar retornos esperados.	Pasa
TP-04	Carga	Observación de comportamiento bajo re-inyección de redeploy (sin pruebas de carga automatizadas).	Observado

3.4. Cobertura y Calidad

Los tests backend se ejecutan en CI con `pytest tests/` y se ejecutan pruebas frontend con `flutter test`. En este proyecto, no se reporta un porcentaje de cobertura mediante `pytest-cov` en el workflow, por lo que la cobertura se evalúa principalmente por el estado de ejecución de pruebas y la calidad de casos cubiertos.

4. Despliegue y Operación

4.1. Infraestructura de Producción

Elemento	Detalle
Plataforma	Servidor universitario (stack desplegado con Docker Compose; Dokploy para auto-deploy en rama <code>main</code>).
Base de datos	MongoDB (contenedor Docker; persistencia vía volumen <code>mongo_data</code>).
Contenedores	Docker Compose: servicios <code>mongo</code> , <code>mosquitto</code> , <code>backend</code> , <code>frontend</code> .
CI/CD	GitHub Actions (tests) + Dokploy (auto-deploy vía webhook) y/o despliegue por SSH con <code>docker compose up -d --build</code> .
Dominio	<code>http://200.27.101.243</code> (Frontend: puerto 8884; API: puerto 8883; MongoDB: puerto 8885; MQTT: puerto 1883).

4.2. Proceso de Despliegue

El flujo de despliegue se basa en:

- **CI:** cada push a `main` ejecuta `pytest` (backend) y `flutter test` (frontend).
- **Deploy:** Dokploy se configura para consumir `docker-compose.yml` y reconstruir el stack, levantando los servicios en el servidor.
- **Alternativa por emergencia:** acceso SSH al servidor y ejecucion de `docker compose up -d --build`.

4.3. Monitoreo y Logs

El backend incorpora middleware que:

- asigna un `request_id`,
- mide latencia por request,
- persiste **logs estructurados** en MongoDB (coleccion de logs) tanto para operaciones exitosas como excepciones no controladas.

Los logs se almacenan en BD de logs con base `cleanpool_logs` y coleccion `api_request_logs`. Esto permite revisar el comportamiento del sistema y detectar errores o tiempos de respuesta anormales.

5. Gestion Scrum y Trabajo Colaborativo

5.1. Estado Final del Sprint

Sprint evaluado: **Sprint 4** (Integracion final del sistema). En esta entrega se consolidan historias asociadas a autenticacion, gestion de piscinas, ingestion IoT, evaluacion de estado y mantenciones.

Objetivo principal: Integrar completamente el flujo sensor → evaluacion → estado en la aplicacion, asegurando un despliegue estable y repetible.

Historias completadas: **4** (HU-01 a HU-04). Historias pendientes: **0** para el alcance de Hito 3. Resultado general: **aprobado para entrega** (segun ejecucion de pruebas CI y validaciones funcionales guiadas).

5.2. Commits y Participacion

Integrante	Aporte principal	Commits (recientes)
Vicente Araya (PM)	Integracion backend-frontend, soporte a despliegue (Dokploy) y mejoras en la ingestion/evaluacion.	30

Diego Carmona (SM)	Implementacion y mejoras de interfaz/funcionalidades en el frontend e integraciones asociadas.	5
Daniel Cortez	Ajustes y mejoras especificas en features de frontend asociadas al sprint.	5
Constanza Malebran	Refinamientos UI y contenido de pantallas (perfil/historial) durante sprint 4.	3

5.3. Problemas y Resoluciones

Problema: conflictos al redeploy (contenedores con nombre ya en uso) y/o permisos en Docker al ejecutar operaciones desde Dokploy.

Resolucion: se unifico el flujo de despliegue para usar un script previo de paro/kill del stack y se documentaron comandos SSH de emergencia (segun `deploy/DOKPLOY.md`) para asegurar reproducibilidad.

Problema: inconsistencias en la evaluacion del estado cuando no existen lecturas MQTT recientes.

Resolucion: se centralizo la logica con una ventana de frescura de **2 minutos** para decidir si el sensor esta ONLINE o si se utiliza mantencion manual mas reciente.

6. Demostracion de Valor

6.1. Valor Entregado al Usuario

El usuario recibe un estado de piscina con base en informacion mas confiable (sensor dentro de ventana) y un respaldo con mantencion manual cuando el sensor no presenta datos recientes. Esto habilita decisiones de mantenimiento mas consistentes y reduce la dependencia de chequeos manuales aislados.

6.2. Evidencia Funcional

Se adjuntan capturas del funcionamiento del sistema: autenticacion, gestion de piscinas, consulta de estado y secciones relacionadas con mantenciones/perfil (ver Anexo A).

- Evidencia de flujo funcional en la aplicacion.
- Confirmacion visual de actualizaciones en estado y vistas del usuario.

6.3. Retroalimentacion Obtenida

La validacion realizada por el equipo y a traves del manual de pruebas indica que los endpoints y el comportamiento esperado se cumplen (registro/login, CRUD, status y mantenciones). Se recomienda continuar optimizando la experiencia de usuario (refinamiento visual) y ampliar evidencia de pruebas de carga automatizadas en futuros hitos.

7. Conclusiones y Proximos Pasos

7.1. Conclusiones Tecnicas

Se consolido una arquitectura integrada con ingestion IoT, persistencia en MongoDB, evaluacion de estado sensor >manual y despliegue reproducible con Docker Compose. La incorporacion de logs estructurados aporta trazabilidad para operacion y soporte.

7.2. Aprendizajes del Equipo

Se aprendio a mantener consistencia entre rutas oficiales y legacy, a formalizar validaciones por ventana temporal (online/stale) y a documentar el flujo de despliegue para evitar fallas por colisiones de contenedores.

7.3. Trabajo Futuro

1. Incorporar pruebas de carga automatizadas y monitoreo mas granular (p. ej., dashboards).
2. Mejorar cache/optimizar consultas criticas (si aplica) para reducir latencia en lecturas frecuentes.
3. Ampliar el soporte de evidencias (capturas y casos) para demos y evaluaciones futuras.

A. Capturas del Sistema

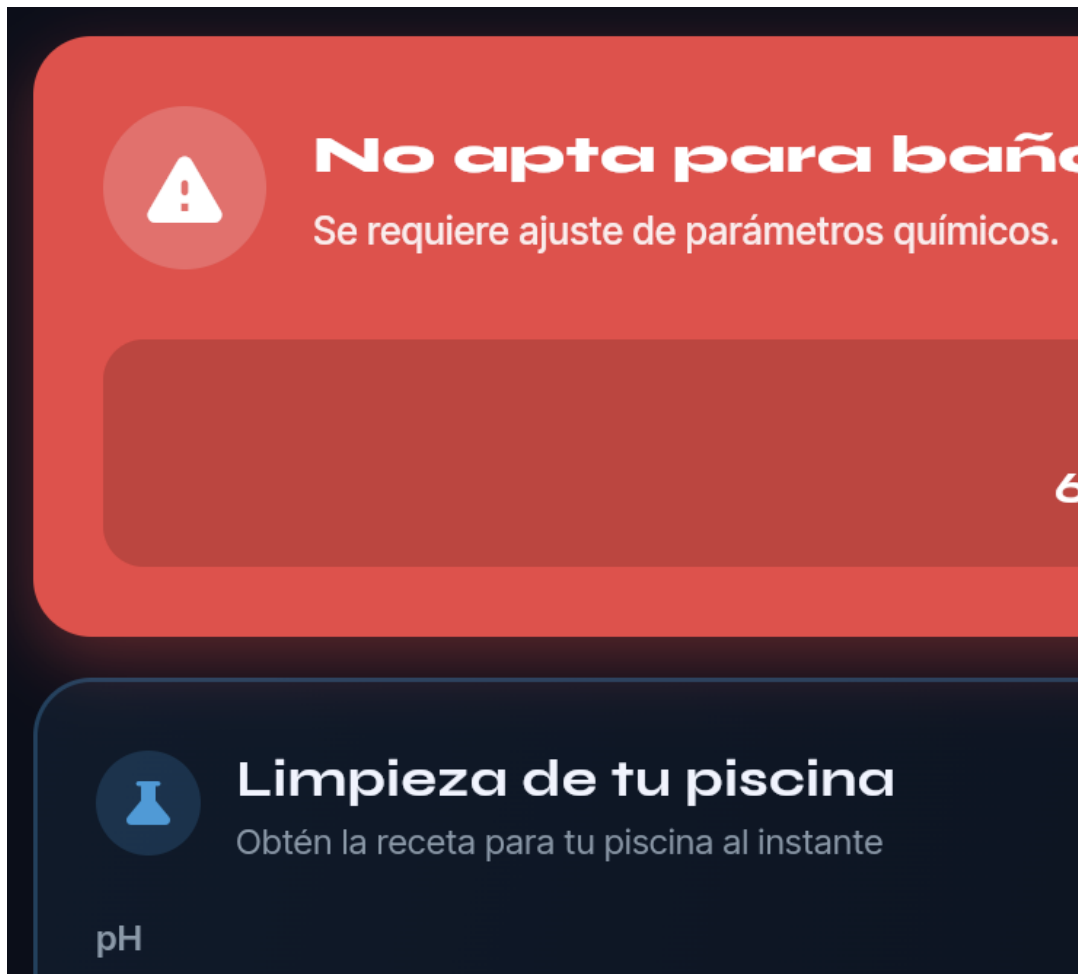


Figura 1: Evidencia de flujo funcional 01

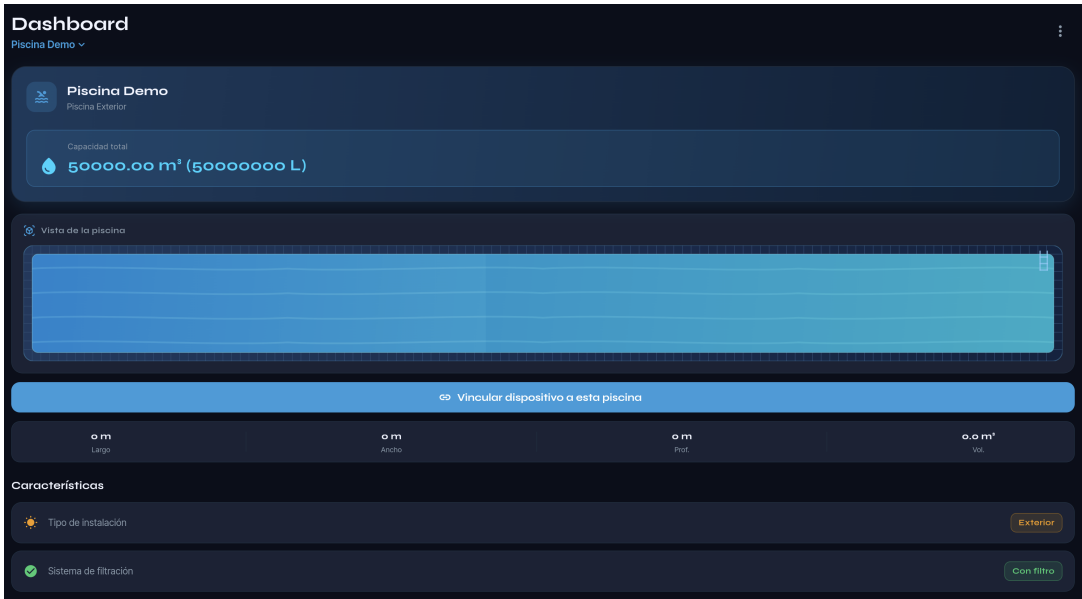


Figura 2: Evidencia de flujo funcional 02

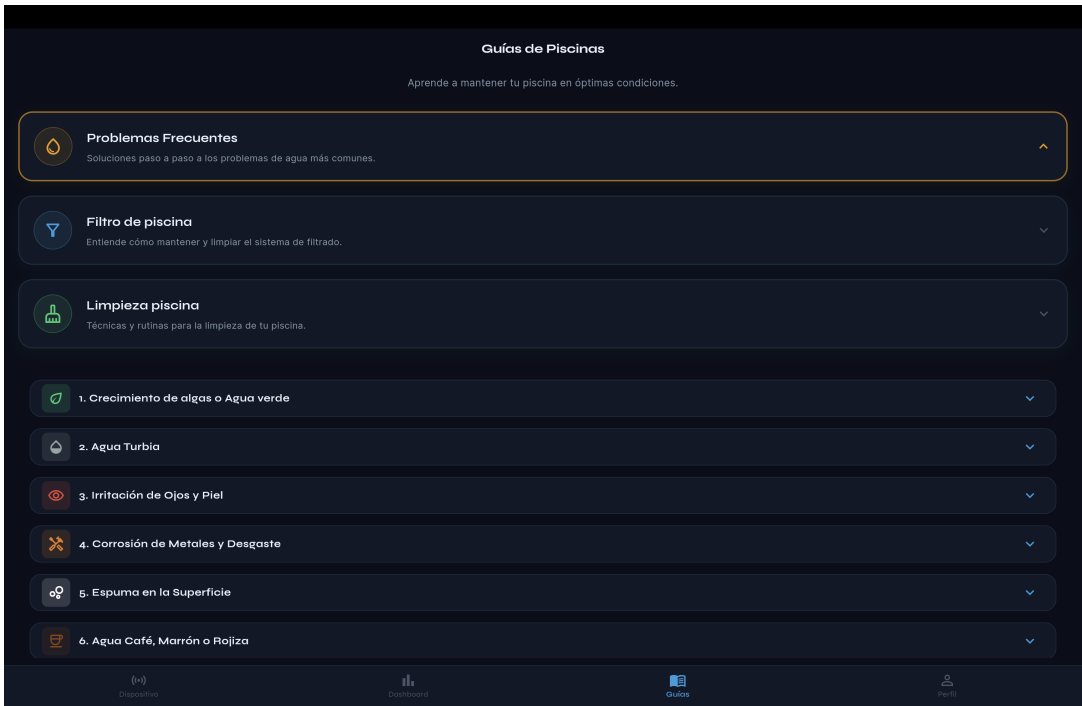


Figura 3: Evidencia de flujo funcional 03

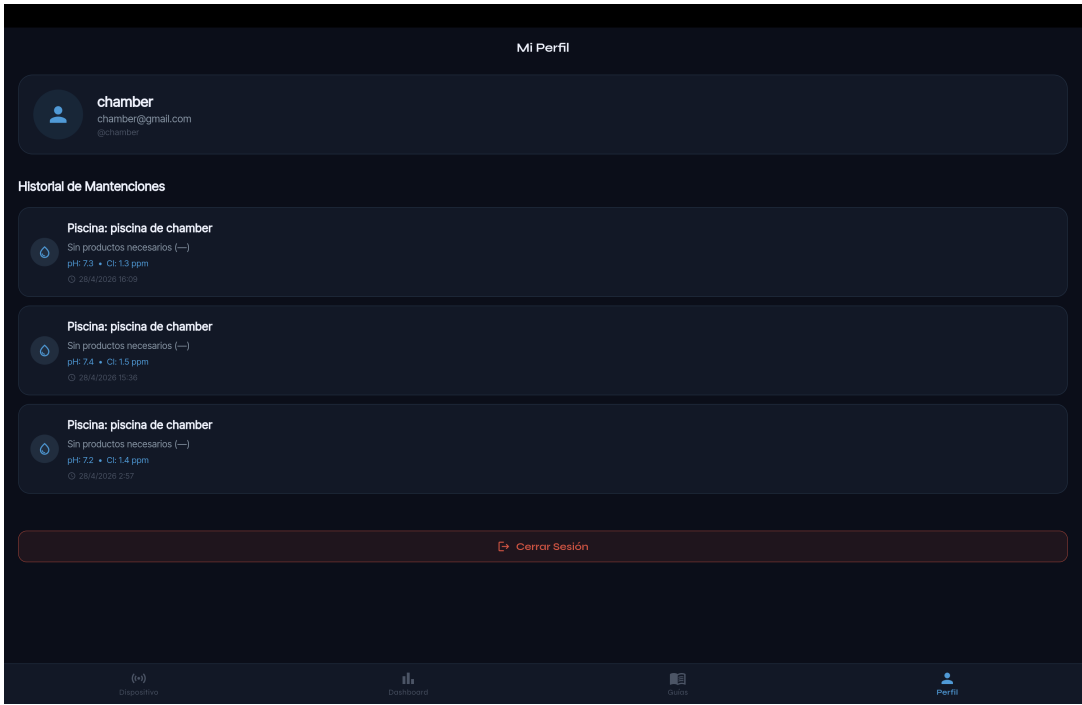


Figura 4: Evidencia de flujo funcional 04

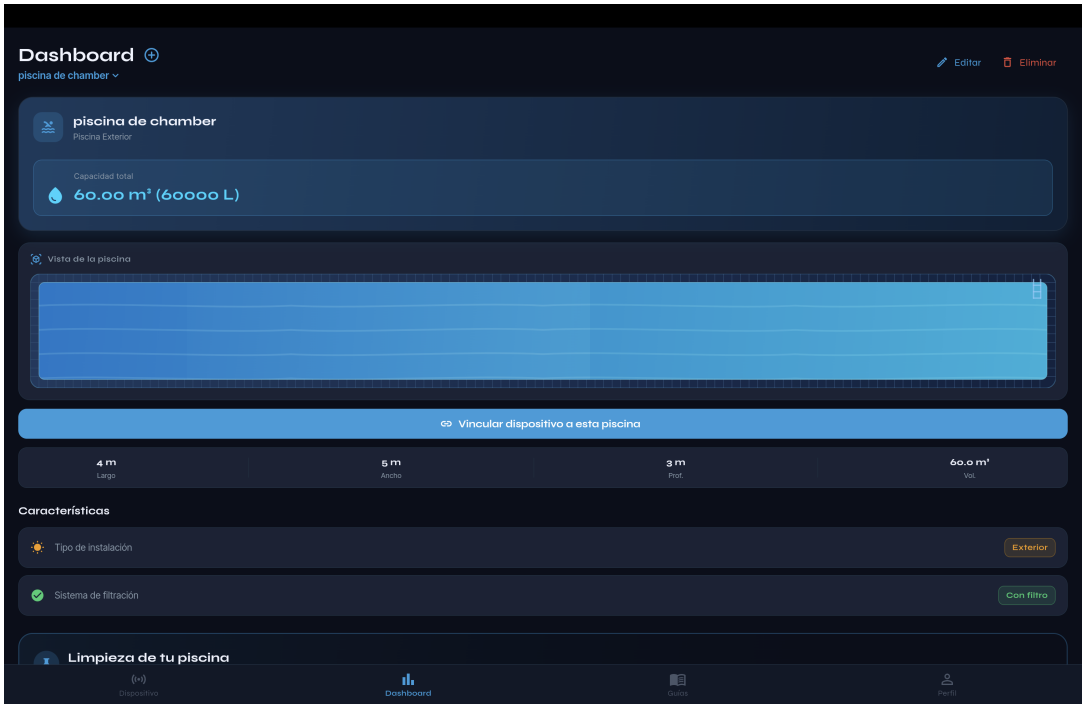


Figura 5: Evidencia de flujo funcional 05

B. Repositorio y Evidencias

Recurso	URL
Repositorio	https://github.com/vicentearaya/software2
Jira/Trello	(Pendiente / no utilizado en esta entrega)
Produccion	http://200.27.101.243:8884

C. Commits Relevantes

1. 3a7b99c: lecturas sensor orp en app
2. 88fb2fc: cambio inicio
3. 294b060: botones para volver a inicio y al cerrar sesion volver inicio
4. 48990e2: prueba de redeploy sin eliminar contenedores
5. 79c9b05: prueba deploy
6. e6604fe: probando redeploy
7. 28c33f9: docs: prueba autodeploy
8. 9c5ceda: fixes para autodeploy con dokploy